

This is an exceptional upgrade from a cloud-dependent Blynk setup to a localized, enterprise-grade Home Assistant ecosystem. You are migrating from a toy architecture to a highly robust, isolated network by utilizing ESP-NOW and direct serial UART. Based on your design intent to distribute the sensors (Kitchen Fire Node) while keeping a central Emergency Node and bridging everything to the Raspberry Pi 5 via a Serial Gateway, here is the complete, professional blueprint.

Phase 1: Architectural Topography & Component Selection

We are building a 3-Node Architecture:

1. **The Serial Gateway Node (The Bridge):** Connects to the Pi 5 via UART GPIO pins.
2. **The Central Emergency Hub (The Siren):** Mains-powered, battery-backed hub in the middle of the house. Handles Earthquake detection and the physical siren.
3. **The Kitchen Fire Node (The Sentinel):** Low-power edge node located in the kitchen. Detects Smoke and extreme Heat.

Component Procurement List

- **Microcontrollers:**
 - 3x ESP32-C3 SuperMini (RISC-V architecture, ultra-low power footprint).
- **Central Emergency Hub Sensors & Actuators:**
 - **Vibration/Earthquake:** SW-420 Module.
 - **Siren/Alarm:** 120dB 12V Active Siren (Do NOT use a standard 5V buzzer, it is not loud enough for a whole home. You will need an NPN transistor like a TIP120 to drive it from the ESP32).
- **Kitchen Fire Node Sensors:**
 - **Smoke/Gas:** MQ-2 Sensor.
 - **Temperature/Humidity:** AHT20 or SHT31 (Discard the DHT11; it is notoriously inaccurate and slow. I²C sensors are much more reliable for life-safety).
- **Power Subsystem (For Central Hub):**
 - **Mains Converter:** Hi-Link HLK-PM01 (220V AC to 5V DC).
 - **Safety Required:** 10D561K Varistor, 0.5A Slow-blow fuse.
 - **Battery Backup:** LiFePO4 (3.2V) cell with a TP5000 charging module (Do not use TP4056, as it is for standard 3.7V Lithium-Ion).
 - **Load Sharing:** P-Channel MOSFET (e.g., NDP6020P) and a Schottky diode (1N5819) to seamlessly switch to the battery during a blackout without resetting the ESP32.

Phase 2: Hardware Assembly & Safety Protocols

The Central Emergency Hub (Mains + Battery)

This node is critical because it integrates 220V AC power.

1. **AC-DC Conversion:** Wire the 220V Live line through the 0.5A fuse, then parallel the 10D561K Varistor across Live and Neutral *before* they enter the HLK-PM01 module. This prevents high-voltage transients from causing the HLK-PM01 to catch fire.
2. **Driving the Siren:** The ESP32-C3 cannot power a 12V siren. Connect the siren's positive

terminal to a 12V source (or use a step-up boost converter from your 5V line). Connect the siren's ground to the Collector of a TIP120 transistor. Connect the Emitter to Ground. Connect the ESP32 GPIO pin to the Base of the transistor via a 1kΩ resistor.

3. **Vibration Sensor (SW-420):** Connect VCC to 3.3V, GND to GND, and D0 to an ESP32 GPIO pin (e.g., GPIO 3).

The Kitchen Fire Node (Low Power)

1. **MQ-2 Sensor:** Connect VCC to a stable 5V rail (the MQ-2 heater draws significant current). Connect GND to GND, and D0 to an ESP32 GPIO pin (e.g., GPIO 4).
2. **AHT20/SHT31 (Temperature):** Connect VCC to 3.3V, GND to GND. Connect SDA and SCL to the ESP32 I²C pins (usually GPIO 8 and GPIO 9 on the C3).

Phase 3: Sensor Calibration (Anti-False Alarm Engineering)

Life-safety systems must not false alarm. Physical calibration is mandatory.

- **SW-420 (Earthquake):**
 1. Place the sensor exactly where it will be permanently mounted (to account for natural floor vibrations).
 2. Turn the blue potentiometer clockwise until the LED stays completely OFF.
 3. Tap the wall/floor lightly. If it triggers, it is too sensitive. Turn the screw slightly counter-clockwise until only a firm shake of the structure triggers the LED.
- **MQ-2 (Smoke/Gas):**
 1. **Burn-in:** The MQ-2 *must* be left powered on for 24-48 hours continuously before calibration. The internal chemical heater needs to bake off manufacturing residues, otherwise it will drift randomly.
 2. After burn-in, turn the blue potentiometer until the signal LED is OFF in clean air.
 3. Light a match, blow it out, and let the smoke waft near the sensor. The LED should trigger.

Phase 4: ESP-NOW Firmware Architecture

The ESP-NOW protocol requires all nodes to share a strict data structure.

1. The Shared Data Structure

All three ESP32s must have this exact structure defined at the top of their code:

```
C++
typedef struct struct_message {
    uint8_t device_id; // 1 = Central Hub, 2 = Kitchen
    bool earthquake_alert;
```

```

bool fire_alert;
bool gas_alert;
float temperature;
} struct_message;
struct_message myData;

```

2. The Kitchen Node Firmware (The Sender)

This node continuously reads the AHT20 and the MQ-2. If temperature > 50 OR MQ-2 == LOW, it sets the flags to true and broadcasts the data via esp_now_send to the Central Hub and the Serial Gateway.

3. The Central Hub Firmware (Sender & Receiver)

This node reads its own SW-420 vibration sensor.

- **If SW-420 triggers:** It blasts the siren (3 continuous beeps) and sends an ESP-NOW packet with earthquake_alert = true to the Serial Gateway.
- **If it receives an ESP-NOW packet from the Kitchen Node:** It checks the packet. If fire_alert == true or gas_alert == true, it immediately blasts the siren (5 continuous beeps) without needing the Pi's permission.

4. The Serial Gateway Node (The Bridge)

This node does nothing but listen. When an ESP-NOW packet arrives, it immediately prints a formatted string to the Hardware Serial port.

```

C++
// Inside the OnDataRecv callback:
void OnDataRecv(const uint8_t * mac, const uint8_t *incomingData, int len) {
    memcpy(&myData, incomingData, sizeof(myData));

    // Format: "ALARM,DEVICE_ID,EARTHQUAKE,FIRE,GAS,TEMP"
    Serial.print("ALARM,");
    Serial.print(myData.device_id); Serial.print(",");
    Serial.print(myData.earthquake_alert); Serial.print(",");
    Serial.print(myData.fire_alert); Serial.print(",");
    Serial.print(myData.gas_alert); Serial.print(",");
    Serial.println(myData.temperature);
}

```

Phase 5: Raspberry Pi 5 & Home Assistant Integration

Because your USB ports are full, you will use the physical GPIO pins on the Pi 5.

Step 1: Physical UART Connection

1. Connect **TX** on the Serial Gateway ESP32 to **RXD** (Pin 10 / GPIO 15) on the Pi 5.
2. Connect **RX** on the Serial Gateway ESP32 to **TXD** (Pin 8 / GPIO 14) on the Pi 5.
3. **CRITICAL:** Connect a GND pin on the ESP32 to a GND pin on the Pi 5 to establish a common voltage reference.

Step 2: Enable Pi 5 UART

1. SSH into your Pi and run `sudo raspi-config`.
2. Navigate to **Interface Options** -> **Serial Port**.
3. Would you like a login shell to be accessible over serial? **NO**.
4. Would you like the serial port hardware to be enabled? **YES**.
5. Reboot the Pi. The serial port will now be available as `/dev/ttyAMA0` or `/dev/serial0`.

Step 3: Home Assistant Configuration

In your `configuration.yaml` (or split sensors file), you will ingest the raw string and disaggregate it.

```
YAML
sensor:
  - platform: serial
    serial_port: /dev/serial0
    baudrate: 115200
    name: "Raw ESPNOW Stream"

template:
  - trigger:
    - platform: state
      entity_id: sensor.raw_espnov_stream
    binary_sensor:
      - name: "Earthquake Detected"
        device_class: vibration
      state: >
        {% set payload = states('sensor.raw_espnov_stream').split(',') %}
        {% if payload | length == 6 and payload[0] == 'ALARM' %}
```

```
{% if payload[2] == '1' %}  
  true  
{% else %}  
  false  
{% endif %}  
{% else %}  
  {{ this.state }}  
{% endif %}
```

You can then create simple Home Assistant Automations that trigger whenever `binary_sensor.earthquake_detected` or `binary_sensor.kitchen_fire_detected` turn on. These automations can use the Telegram integration natively inside Home Assistant to blast messages to your family group, completely replacing the need for CallMeBot!